

Term-project 보고서

컴퓨터프로그래밍3 02분반

202302533 김승우

프로젝트 개요

학생들의 정보를 관리하는 프로그램입니다. 연결 리스트를 통해서 학생의 정보를 관리해 주었고 admin과 client모드를 두어 저장된 정보에 접근할 수 있는 정도에 제한을 두었습니다. admin모드의 경우 모든 접근이 가능하여 삽입 삭제등의 수정을 할 수 있지만 client 모드일 경우에는 오직 읽기 밖에 할 수 없도록 설계를 하였습니다.

프로젝트 설계 과정

학생들의 정보를 구조체 형태로 저장하여 이를 연결리스트로 관리를 해야 했습니다. 가장 먼저 student파일을 구현해 줌으로 학생 구조체를 연결리스트 형태로 만들어 주는 함수들을 설계했습니다. 새로운 학생들이 추가 될때마다 동적할당으로 구조체 크기의 메모리를 할당해 주었고 구조체 포인터를 활용하여 삭제를 효율적으로 할 수 있도록 하였습니다. student파일이 학생들의 연결리스트를 다루는 파일이라면 file_io파일은 csv파일을 다루는 함수들을 구현한 파일입니다. id,name,score형식으로 저장되어 있는 csv파일의 내용을 읽어와 연결리스트로 만들어 주거나 혹은 현재 연결리스트에 저장되어 있는 값들을 csv에 저장해주는 함수들을 구현해 놓았습니다. 마지막으로 command파일을 통해서 입력된 명령어들과 현재 shell에 따라 수행해야 하는 동작들을 함수로 구현해 주었습니다. 연결리스트, csv파일 둘중 어느 곳에 혹은 모두 접근해야 할지 파악하여 명령어에 맞는 함수들을 구현해 주었습니다. main.c파일에서 명령어를 직접 입력할지 혹은 명령어 파일을 사용할 지 정해져 해당 함수를 불러오도록 하였습니다.

명령어 모음

명령어	Admin	Client	설명
list	O	O	전체 학생목록 출력
find <id>	O	O	id로 학생 검색
help	O	O	명령어 도움말 출력
exit	O	O	프로그램 종료
reload	O	O	CSV에서 학생 정보 불러오기
save	O	X	CSV에 학생 정보 저장
add <id> <name> <score>	O	X	학생 추가
delete <id>	O	X	학생 삭제
update <id> <score>	O	X	점수 수정
Client Mode에서는 오직 정보를 읽어오는 것만 가능			

각 파일 별 설명

student.h / student.c

연결 리스트로 이어져 있는 학생 구조체를 선언해 둔 파일입니다. 구조체 생성, 학생 삽입, 학생 탐색, 학생삭제, 모든 학생을 없애기, 학생 수 리턴, 이렇게 총 6가지 함수를 구현하였습니다.

create_student: malloc을 통하여 새로운 학생이 생성될 때 마다 구조체 크기의 메모리를 할당해 주었고 이후 받아온 파라미터들을 통해 학생들의 고유한 값들을 저장해 주었습니다.

insert_student: 연결리스트에 새로운 학생 구조체를 추가해 주는 함수입니다. 리스트의 가장 마지막에 새로운 학생이 추가됩니다.

find_student: 학생의 id를 받아 연결 리스트 내부에 해당 학생이 있는지 탐색하는 함수입니다.

delete_student: 학생의 id를 받아 해당 학생을 삭제하는 함수입니다. 연결리스트에 해당 학생이 존재하여 삭제를 성공한 경우 1, 존재하지 않아 삭제를 하지 못한경우 0을 반환하도록 하였습니다. 삭제한 학생은 free를 사용하여 메모리를 날려줌으로 flush등이 일어나지 않도록하여 메모리 관리를 해주었습니다.

free_students: 연결 리스트를 초기화 하는 함수입니다. 리스트 내부의 모든 학생 구조체가 가지고 있는 메모리를 free()를 이용해 초기화 해주는 것으로 리스트 자체를 없애줍니다.

num_students: 연결리스트의 학생이 총 몇명이 있는지 반환하는 함수입니다.

file_io.h / file_io.c

csv파일에 있는 값을 다루는 함수들을 구현한 파일입니다. csv파일에서 값을 불러오고, 저장하는 함수를 구현하였습니다.

load_students: csv파일에 저장된 학생들을 읽어오는 함수입니다. fopen을 이용하여 입력받은 파일을 열어주고 만약 파일 없는 경우를 처리해 주었습니다. 이후 새로운 학생 구조체와 파일의 각 행을 읽어 올 문자열을 선언하였습니다. csv파일의 가장 윗부분은 참고용 header이기 때문에 fgets를 사용하여 윗부분을 먼저 날려 준 뒤에 반복문을 통해 학생들을 저장해 주는 방법을 생각하였습니다. 한 줄 씩 읽어온 문자열을 토큰화 하여 id, name, score로 분류를 하였고 create_student함수를 이용하여 학생들을 추가해 주었습니다. 모든 학생을 추가한 뒤에는 파일 손상방지와 메모리 관리를 위하여 fclose를 통해 읽고 있던 파일을 닫아 주었습니다.

save_students: 현재 연결 리스트에 저장되어 있는 학생들을 csv파일에 저장하여 값들을 갱신해 주는 함수입니다. 저장할 csv파일의 이름을 받아와 해당 파일에 반복문을 통해 학생들을 저장해 줍니다. 마찬가지로 모든 학생의 저장을 마친 뒤에 fclose를 이용하여 파일을 닫아 주었고, 저장 할 때마다 횟수를 세주어 저장된 인원이 몇명인지 알 수 있도록 하였습니다.

command.h / command.c

shell의 작동 현황을 알려주도록 enum을 통해서 각 상황들을 선언해 주었고, 입력 받을 명령어와 실행할 함수를 바로 알 수 있도록 command구조체를 만들어 명령어, 함수, 명령어 설명을 담아주었습니다. command배열과 입력받은 csv파일 이름을 저장한 문자열이 존재한다는 것을 알려주기 위해 extern을 사용해 주었고 배열의 인자 개수와 문자열 수정을 위한 함수들을 헤더파일에 선언하였습니다. 이후 각 명령어들을 구현하기 위한 함수들을 설계해 주었습니다.

admin과 client 모드일때의 사용가능한 명령어의 종류가 다르기 때문에 ifdef를 이용하여 admin과 client일때의 commands배열 내부 값을 다르게 설정해 주었습니다.

handle_save: 현재 연결 리스트에 저장된 학생들의 정보를 csv파일에 저장해 주는 함수입니다. 연결 리스트의 head부터 시작해서 마지막 학생까지 순서대로 csv파일에 저장을 해 줍니다. 저장을 할때에는 file_io에서 구현해 두었던 save_students함수를 사용하였습니다.

handle_reload: csv파일에 저장되어 있는 학생정보를 연결리스트 형태로 불러오는 함수입니다. 입력받은 csv파일이 비어있을 경우 예외처리를 해두었고 원래

가리키고 있던 연결리스트를 free_students함수를 통해서 없애주어 새로운 학생 정보를 담을 준비를 하였습니다. 이후 csv파일의 첫째 줄 부터 시작하여 값을 연결리스트에 불러오도록 load_students함수를 사용해 주었습니다. 어떻게 처리가 되었는지 메시지를 출력해야 하기 때문에 연결리스트 내부의 학생 수와 csv 파일의 이름을 프린트 하도록 하였습니다.

handle_add: 연결리스트에 새로운 학생을 추가해 주는 함수입니다. 학생 정보를 파라미터로 받아오기 때문에 조건에 맞는지 확인을 먼저 해주었습니다. 3개의 파라미터가 잘 들어왔는지, 점수가 조건 범위에 포함되는지, id가 이미 존재하는 상태인지, id가 0보다 큰지를 확인해 주었습니다. 이후 해당 파라미터들로 create_student함수를 통해 새로운 학생 구조체를 만들고 insert_student함수로 연결리스트에 추가해 주었습니다.

handle_delete: 학생의 id를 받아 해당학생의 정보를 연결리스트에서 없내는 함수입니다. 해당 id가 연결리스트 내부에 존재하는지 확인을 해 주고 delete_student함수를 이용하여 해당 학생을 삭제해 주었습니다.

handle_update: 학생의 id와 score를 파라미터로 받아서 연결리스트 내부의 해당 id를 가진 학생의 점수를 갱신해 주는 함수입니다. 파라미터 개수가 2개가 맞는지, 점수가 조건에 부합하는지 확인을 해주어 먼저 예외처리를 하였습니다. 이후 find_student함수를 통해서 해당 id를 가진 학생을 새로운 학생 구조체 포인터 s에 저장해 주었고 학생이 없는 경우 예외처리, 존재 하는경우 점수를 갱신해 주었습니다.

handle_find: 학생의 id를 파라미터로 받아와 그 학생의 정보를 출력하는 함수입니다. 파라미터의 개수가 맞게 들어왔는지, 연결리스트 내부에 해당 id가 존재하는지 먼저 확인을 해주고 그렇지 않을 경우 예외처리를 하였습니다. 이후 구조체 포인터를 통해서 해당 학생의 정보들을 출력하도록 해주었습니다.

hadle_list: 연결 리스트에 있는 모든 학생들의 정보를 출력하는 함수입니다. 리스트 내부가 비어있는 경우 예외 처리를 해주었고 포인터를 통해서 head부터 마지막까지 나아가면서 학생들의 정보를 출력해 주었습니다.

handle_stats: 연결리스트 내부에 있는 학생들의 정보를 종합하여 학생 수, 최댓값, 최솟값, 평균을 출력하는 함수입니다. 리스트 내부가 비어있는 경우 예외처리를 해주었고 최댓값을 -1로 초기화, 최솟값을 101로 초기화 한 상태로 학생들과 계속 비교를 해 나가며 값을 구해주었습니다.

handle_help: 사용가능한 명령어와 사용방법을 출력하는 함수입니다. commands 배열 내부에 있는 값들을 차례대로 출력해주었습니다. admin상태라면 11개의 명령어를, clinet상태라면 7개의 명령어를 보여주도록 설계하였습니다.

handle_clear: 터미널 화면을 지워주는 함수입니다. pdf자료에 나와있는 내용을

활용하여 구현해 주었습니다.

handle_exit: 프로그램을 종료하는 함수입니다. goodbye를 출력하고 enum인 SHELL_EXIT을 리턴하여 main.c파일에서 해당 결과를 받을 경우 프로그램을 종료하도록 설계하였습니다.

main.c

앞에 말한 모든 헤더파일을 include하고 있고 프로그램의 동작을 구현하고 있습니다. 명령어를 직접 작성하는 경우와, 명령어 파일을 읽어오는 경우를 나누어서 함수를 설계 하였습니다.

main: default csv파일로 "students.csv"를 설정해 두고 입력된 파일 이름으로 다시 가리키는 csv파일을 초기화해 주었습니다. argv에 입력된 문자열을 읽어 -f가 첫부분에 있다면 명령어 파일을 읽어오는 프로그램을 실행하고 그렇지 않은 경우 주어진 csv파일을 바탕으로 직접 명령어를 작성하는 프로그램을 실행하도록 하였습니다. ifdef를 이용하여 admin, client 각 두 모드일때 알맞는 shell을 만들어 프로그램이 동작하도록 해주었습니다. 명령어 파일이 존재할 경우 해당 파일과 csv파일을 파라미터로 전달해 주고 그렇지 않은 경우 csv파일만 파라미터로 전달하였습니다.

run_shell: 직접 명령어를 작성하는 프로그램을 구현한 함수입니다. 프로그램이 시작할 때 어떤 모드인지 알려주기 위해 ifdef를 사용하여 어떤 프로그램인지 프린트 하여 알려주도록 하였습니다. 파라미터로 받은 csv파일의 이름을 통해서 load_student함수로 학생정보를 담은 연결리스트를 만들어 주었습니다.

command파일에서 구현한 reset_csvName함수를 통해서 default인 students.csv 파일을 가리키고 있는 command내부의 함수들이 주어진 csv파일을 가리킬 수 있도록 해주었습니다. 이후 반복문을 통해서 exit명령어가 들어올 때 까지 계속해서 문장을 받아 처리해 주었습니다. 입력받은 문장은 strtok함수를 이용하여 토큰화를 해주었고 명령어 부분과 그외 부분으로 나누어 주었습니다. 이후 해당 명령어를 찾아 주어진 명령을 실행해 주었고 exit명령이 들어와 SHELL_EXIT을 리턴 받았을 경우에 free_students함수로 연결리스트의 메모리를 해제해 주고 프로그램을 종료하였습니다. 또한, 명령어가 올바르지 않으면 Error메세지를 출력하도록 하였고 아무것도 입력되지 않은 경우 반복문을 break한 뒤에 free_students함수를 통하여 리스트에 할당된 메모리는 해제해 주었습니다.

run_command_file: 로직은 run_shell과 같지만 명령어를 사용자가 입력하는 것이 아닌, 명령어 파일을 읽어오는 형식으로 프로그램이 동작합니다. 읽어온 명령어 파일이 비었다면 Error메세지 출력 후 프로그램이 종료됩니다. 정상 동작을 하는 경우 명령어 파일을 한줄 씩 읽어오며 run+shell과 같은 동작을 하도록 구

현을 하였습니다. 각 명령이 몇번째 인지 횟수를 세주며 결과에 command file N번째 형식으로 알 수 있도록 해주었고 잘못된 입력의 경우 skipped을 진행하는 메시지가 출력되도록 해주었습니다. exit이 들어오거나 모든 줄을 읽어서 프로그램을 종료해야 하는 경우 free_students를 실행하고 또한 fclose를 실행하여 읽어들인 명령어 파워를 닫아 줌으로서 메모리 관리를 해주었습니다.

Test Case 결과

admin shell – 직접 명령어 입력

초기 student.csv파일 내용.

test 이후 student.csv파일 내용.

students

id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70

students

id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70
4	kim	100

```
> make admin
gcc -Wall -Wextra -std=c11 -DADMIN_MODE main.c student.c file_io.c command.c -o admin_shell
> thototh ~/Desktop/term-project-thototh/project & main
> ./admin_shell students.csv
[Admin Program]
Loaded 3 students from students.csv.

admin> help
Commands
save                Save students to CSV
reload              Reload students from CSV
add <id> <name> <score> Add a student
delete <id>         Delete a student
update <id> <score> Update student score
find <id>           Find student
list               List students
stats              Show statistics
help              Show help
clear             Clear screen
exit              Exit shell

admin> add 4 kim 100
Student added
admin> save
Saved 4 students to students.csv.
admin> reload
Reloaded 4 students from students.csv.
admin> list
ID      Name      Score
1       Alice      50
2       Bob        60
3       Charlie    70
4       kim         100
admin> update 1 88
Student updated.
admin> list
ID      Name      Score
1       Alice      88
2       Bob        60
3       Charlie    70
4       kim         100
admin> delete 3
Student deleted.
admin> list
ID      Name      Score
1       Alice      88
2       Bob        60
4       kim         100
admin> exit
GoodBye.
```

client shell – 직접 명령어 입력

초기 student.csv파일 내용.

test 이후 student.csv파일 내용.

students		
id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70
4	kim	100

students		
id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70
4	kim	100

```
thototh ~ /Desktop/term-project-thototh/project
> make client
gcc -Wall -Wextra -std=c11 -DCLIENT_MODE main.c student.c file_io.c command.c -o client_shell
thototh ~ /Desktop/term-project-thototh/project
> ./client_shell students.csv
[Client Program]
Loaded 4 students from students.csv.

client> help
Commands
reload                Reload students from CSV
find <id>             Find student
list                 List students
stats               Show statistics
help               Show help
clear             Clear screen
exit             Exit shell

client> add 1 name 100
Error: Unknown command or permission denied.
client> stats
Count: 4
Average: 70.0
Max: 100
Min: 50
client> list
ID      Name    Score
1       Alice   50
2       Bob    60
3       Charlie 70
4       kim    100
client> delete 1
Error: Unknown command or permission denied.
client> find 4
ID: 4
Name: kim
Score: 100
client> eixt
Error: Unknown command or permission denied.
client> exit
GoodBye.
thototh ~ /Desktop/term-project-thototh/project
```


Admin, Client에서 동일하게 실행할 명령어 파일

```
≡ commands.txt
1 list
2 help
3 add 7 park 55
4 save
5 stats
6 delete 7
7 exit
```

admin shell – 명령어 파일 입력

초기 student.csv파일 내용. test 이후 student.csv파일 내용

students

id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70
4	kim	100

students

id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70
4	kim	100
7	park	55

```
> make admin
gcc -Wall -Wextra -std=c11 -DADMIN_MODE main.c student.c file_io.c command.c -o admin_shell
> thototh ~ /Desktop/term-project-thototh/project main
> ./admin_shell students.csv -f commands.txt
[Admin Program]
Loaded 4 students from students.csv.

[command file:1] list
ID      Name    Score
1       Alice   50
2       Bob    60
3       Charlie 70
4       kim    100

[command file:2] help
Commands
save                Save students to CSV
reload              Reload students from CSV
add <id> <name> <score> Add a studnet
delete <id>         Delete a student
update <id> <score> Update studnet score
find <id>           Find student
list               List students
stats              Show statistics
help              Show help
clear             Clear screen
exit             Exit shell

[command file:3] add 7 park 55
Student added

[command file:4] save
Saved 5 students to students.csv.

[command file:5] stats
Count: 5
Average: 67.0
Max: 100
Min: 50

[command file:6] delete 7
Student deleted.

[command file:7] exit
GoodBye.
```

client shell – 명령어 파일 입력

초기 student.csv파일 내용. test 이후 student.csv파일 내용

students		
id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70
4	kim	100
7	park	55

students		
id	name	score
1	Alice	50
2	Bob	60
3	Charlie	70
4	kim	100
7	park	55

```
> make client
gcc -Wall -Wextra -std=c11 -DCLIENT_MODE main.c student.c file_io.c command.c -o client_shell
> thototh ~/Desktop/term-project-thototh/project main ±
> clear
> thototh ~/Desktop/term-project-thototh/project main ±
> ./client_shell students.csv -f commands.txt
[Client Program]
Loaded 5 students from students.csv.

[command file:1] list
ID      Name    Score
1       Alice   50
2       Bob     60
3       Charlie 70
4       kim     100
7       park    55

[command file:2] help
Commands
reload          Reload students from CSV
find <id>       Find student
list            List students
stats          Show statistics
help           Show help
clear          Clear screen
exit           Exit shell

[command file:3]
Error: unknown commandSkipped line 3

[command file:4]
Error: unknown commandSkipped line 4

[command file:5] stats
Count: 5
Average: 67.0
Max: 100
Min: 50

[command file:6]
Error: unknown commandSkipped line 6

[command file:7] exit
GoodBye.
```

템프로젝트 과정 중 AI사용 내역

프로젝트를 시작하는 단계에서 pdf에 쓰여진 여러 파일 들 중 어떠한 순서로 구현을 하는것이 맞는지 파악을 하지 못하였습니다. main.c를 가장 먼저 설계해야 한다고 생각하고 계속 어떻게 코드를 짜면 될지 계속 생각하던 중 ai에게 파일 구현 순서를 물어보았습니다. student, file_io, command, main, makefile순으로 프로그래밍을 하라는 답변을 받았고 어떠한 이유로 해당 순서대로 코드를 짜야 하는지 이해할 수 있어서 이후 구현에 도움이 되었습니다.

명령어 파일을 읽어오는 함수를 구현할 때 java의 BufferedReader, StringTokenizer 메서드와 유사한 기능을 가진 함수를 필요로 했습니다. 구글 검색을 통하여 str라이브러리에 해당 함수가 있다는 것을 알게되었지만 구체적인 사용방법이 계속 헛갈려 ai에게 해당 부분의 구현을 부탁하였습니다.

```
//strtok 문자열을 토큰화 해서 나누어 주는 메서드, CSV파일에 id,name,score로 저장됨
char* id_s = strtok(line, ",");
char* name = strtok(NULL, ",");
char* score_s = strtok(NULL, ",");

int id = atoi(id_s);
int score = atoi(score_s);
```

(file_io.c)

한계점과 개선방향 및 느낀점

명령어 파일을 두개 이상 입력할 경우 오류가 발생하여 실행이 되지 않는 코드의 단점이 있습니다. 또한, 학생들의 순서를 정렬하는 함수를 시간이 부족하여 만들지 못해 부족함을 남긴 것 같습니다. 전체적인 코드를 구현하면서 하나의 프로그램을 만들때 어떠한 과정으로, 각 파일별로 어떻게 접근이 이루어 지는지 이해 할 수 있게 되는 과제였습니다. 채점기를 이용하여 마지막 확인을 하던중 save가 제대로 이루어 지지 않음을 발견 하였고, 과제 자체를 잘못 이해했다는 것을 알았습니다. 학생들을 항상 students.csv파일에 저장해야 한다고 생각하였기에 틀린 답이 나왔다는 것을 찾았고 이후 이를 수정해 주면서 주어진 조건을 만족 시킬 수 있었습니다.